脓毒症可信治疗决策的记忆演化机制研究

1. 脓毒症多模态记忆的生理约束表征机制

该部分要构建面向脓毒症治疗场景的神经符号融合记忆本体。在这里，为了存储记忆，我们构建以下结构来分别存储患者信息，医生反馈信息以及专业知识。

1.1 患者信息

患者信息部分的存储内容是患者的基本信息，如年龄病史等，以及入院后的生理指标变化。对于患者信息 Patient（n 个患者，可动态增加）：总体的患者信息由一个列表储存，列表内的每个元素对应一个患者的索引（在表一中由 1-n 代替，实际为患者 ID），每一个索引指向表 2。

表 1：患者列表

1	2	3	.....	n

↓

表 2：患者 A 的信息储存示例 PatentRecord

t	State (基本信息和后续监测指标，如 Age，MAP，SOFA 评分相关指标)			
0				
1				
2				
.....				
m				

患者的信息储存形式为 Class （PatientRecord），在该类中存在以下字段：

- embedding：存储多模态时序编码器输出的患者嵌入向量（后续应用于多模态时序编码器中的 patient_embedding）；

- concept_activations：存储临床概念激活度（后续应用于多模态时序编码器中的 concept_activations）。
- State 结构体：保存患者基本信息以及后续监测的生理指标。其中会包含患者的疾病诊断（疾病名储存在疾病库中）

患者信息可执行增加，修改等操作。

1.2 医生决策修正轨迹

该部分储存有关医生以及诊疗的信息。对于每个 Doctor，使用一个 class（DoctorProfile）的方式存储，存储内容包括个人信息，干预方案以及决策修正轨迹。

医生个人信息（personal_info）为固有信息，可修改但是不会轻易增删内部的变量。包括以下信息：

- 专长（疾病名，储存在疾病库中）
- 职位（不同职务有不同的基础置信度）
- 成功率（可动态更新，每次更新其诊断记录都会更新）

干预方案 Prescription 的储存方式与患者类似，也是列表+索引。在每条干预方案中，包含以下信息：

- 患者信息 Patient，直接指向对应患者的 PatentRecord 类。
- 干预措施 Inv，包含以下内容
  1) 采用药物 Med （存储在药物库中）
  2) 使用方式 Op （存储在操作库中）
  3) 使用时间 t_med （对应到患者信息中的相对时间）

决策修正轨迹 decision_trajectory，存储决策历史日志 FeedbackLog，每个日志条目包含反馈 ID、医生 ID、患者 ID、干预类型、时间戳、反馈结果。时间戳为日志的修改时间，存储方式与医生个人信息与干预方案类似，干预方案部分要链接到对应的 Prescription。决策历史日志 FeedbackLog 的存储方式代码如下：

```python
class FeedbackLog:
    def __init__(self):
        self.events = []

    def add_event(self, event):
        self.events.append(event)

    def get_similar_feedbacks(self, intervention, diagnosis):
        # 返回同类干预和诊断的反馈事件
        return [e for e in self.events if e.intervention == intervention and e.patient_diagnosis == diagnosis]

class FeedbackEvent:
    def __init__(self, event_id, doctor_id, patient_id, intervention, timestamp, outcome, knowledge_impact=None):
        self.event_id = event_id
        self.doctor_id = doctor_id
        self.patient_id = patient_id
        self.intervention = intervention
        self.timestamp = timestamp
        self.outcome = outcome
        self.knowledge_impact = knowledge_impact
```

1.3 知识存储

该部分主要存储规则的范式（使用时序逻辑的方法储存）以及知识图谱组件。规则范式存储方式为 clinical_rules 列表，每个规则用符号逻辑表示：

- 在状态 S(i)时，使用方案 Inv，在 t 时间后，转变为状态 S(j) $S_i \xrightarrow{\text{Inv}} \bigcirc\bigcirc S_j$
- 状态 S(i)，其边界为 70 ~ 140 $S_i(0) \rightarrow (70\sim140)$
- 在状态 S(i)时，则患者 Patient 确诊脓毒症 $S_i \rightarrow \text{Sepsis}$

指标名称存储库，疾病名称库，药物库，操作方式库等等，这些均设置成类。知识存储部分结果示例如下：

```python
class MedicalKnowledgeBase:
    def __init__(self):
        self.clinical_rules = []  # 规则列表，例如 [{'condition': 'S_i', 'intervention': 'Inv', 'outcome': 'S_j'}]
        self.disease_library = DiseaseLibrary()  # 疾病库
        self.medication_library = MedicationLibrary()  # 药物库
```

```python
        self.procedure_library = ProcedureLibrary()  # 操作库
        self.symbolic_constraints = []  # 符号约束规则

    def add_rule(self, rule_dict):
        # 一致性验证
        if self.validate_rule(rule_dict):
            self.clinical_rules.append(rule_dict)
        else:
            raise ValueError("Rule violates symbolic constraints")

    def validate_rule(self, rule_dict):
        # 验证逻辑
        intervention = rule_dict.get('intervention')
        if intervention not in self.medication_library.get_all_medications():
            return False
        return True
```

2. 记忆引导的反事实治疗推演机制

记忆引导的反事实治疗推演机制的流程始于多模态时序编码器对查询患者数据进行编码，生成患者信息的结构化表示；通过联邦元学习匹配器在隐私保护下搜索相似候选患者；接着基于 LLM 的反事实预测模块预测相似患者的干预措施对当前患者的反事实效果，并通过神经符号相似性度量进行精细分析，计算数值、符号和因果维度的相似性，最终结合临床目标生成综合评分和可解释报告。下面是各个模块的实现：

2.1 多模态时序编码器

多模态时序编码器的作用是将患者的生命体征、实验室结果和临床事件等原始时序数据编码为结构化的表示，以支持后续工作。该模块通过集成多尺度一维卷积神经网络 1D-CNN 来捕捉不同时间尺度的生理指标：分钟级卷积→短期模式如心率；小时级卷积→中期模式如血压变化；天级卷积→长期模式疾病进展。将特征对齐并融合后，使用时序注意力机制加权突出干预时刻或病情转折点等临床关键时间点，再通过全局池化生成患者嵌入向量，最后利用概念瓶颈层将嵌入映射到可解释的各种症状及病情发展，从而输出患者嵌入、概念激活度和注意力权重三元组，为类比推理提供基础。伪代码表示如下：

```
# 多模态时序编码器
CLASS MultiscaleTemporalEncoder:
    # 初始化：设置多尺度卷积、注意力机制、概念瓶颈层
    METHOD __init__(input_dim, clinical_concepts):
        # 1D_CNN 一维卷积神经网络
        self.minute_scale_conv = 1D_CNN(kernel=3)    # 分钟级波动
        self.hour_scale_conv = 1D_CNN(kernel=12)      # 小时级趋势
        self.day_scale_conv = 1D_CNN(kernel=24)       # 天级模式
        self.temporal_attention = MultiHeadAttention() # 时序注意力
        self.concept_bottleneck = MLP()               # 临床概念映射层

    # 前向传播：将原始时序数据编码为结构化表示
    METHOD encode(vital_signs, lab_results, clinical_events):
        # 1. 多尺度特征提取
        minute_features = EXTRACT_FEATURES(vital_signs,
```

```
self.minute_scale_conv)
        hour_features = EXTRACT_FEATURES(vital_signs, self.hour_scale_conv)
        day_features = EXTRACT_FEATURES(vital_signs, self.day_scale_conv)

        # 2. 多尺度特征融合与对齐
        aligned_features = ALIGN_AND_CONCAT(minute_features, hour_features, day_features)

        # 3. 时序注意力机制：关注临床关键时间点
        attended_features, attention_weights = APPLY_ATTENTION(aligned_features)

        # 4. 全局池化得到患者级表示
        patient_embedding = GLOBAL_AVERAGE_POOLING(attended_features)

        # 5. 概念瓶颈层：映射到可解释的临床概念
        concept_activations = CONCEPT_BOTTLENECK(patient_embedding)

        RETURN {
            'patient_embedding': patient_embedding,      # 用于相似性匹配的向量
            'concept_activations': concept_activations,   # 可解释的临床概念激活度
            'attention_weights': attention_weights        # 显示哪些时间点最重要
        }
```

2.2 基于 LLM 的反事实预测模块

基于 LLM 的反事实预测模块利用经过医学微调的大型语言模型进行反事实推理，以模拟不同干预措施对患者状态的潜在影响。该模块需要输入当前患者状态描述、干预措施以及类似患者的历史结局数据，调用 LLM 生成反事实预测轨迹。LLM 的输出经过解析和验证，计算其预测置信度。该模块还会量化反事实预测轨迹与实际轨迹的相似性，从而实现反事实通道。

```
# 基于 LLM 的反事实预测
CLASS CausalTemporalGraph_LLM:
    METHOD __init__(clinical_llm, medical_knowledge_base):
```

```
        self.clinical_llm = clinical_llm  # 经过医学微调的大模型
        self.knowledge_base = medical_knowledge_base
        self.causal_reasoner = CausalChainExtractor()

    METHOD predict_counterfactual_with_llm(patient_state, intervention, historical_outcomes):
        """
        使用大模型进行反事实推理
        patient_state: 患者当前状态描述
        intervention: 要评估的干预措施
        historical_outcomes: 类似患者的历史结局数据
        """

        # 1. 构建反事实推理提示
        prompt = f"""
        你是一名资深临床专家。请基于以下信息进行反事实推理：

        患者当前状态：
        {patient_state}

        拟实施干预：
        {intervention}

        参考历史数据（类似患者接受该干预的结局）：
        {historical_outcomes}

        请推理：如果对该患者实施此干预，最可能的生理参数变化轨迹是什么？
        ......

        推理要求：
        ......
        """

        # 2. 调用医学 LLM 进行推理
        llm_response = self.clinical_llm.generate(prompt, max_tokens)

        # 3. 解析和验证 LLM 输出
        structured_prediction = self.parse_and_validate_llm_output(llm_response)

        # 4. 基于医学知识库进行合理性校验
        validated_prediction = self.medical_knowledge_validation(structured_prediction)

```

```
        RETURN validated_prediction

    METHOD counterfactual_similarity_llm(patient_a, patient_b, treatment_options):
        #基于 LLM 反事实预测的相似性计算
        similarities = []

        FOR treatment IN treatment_options:
            # 对患者 A 进行反事实预测
            cf_prediction_a = self.predict_counterfactual_with_llm(
                patient_a.current_state,
                treatment,
                patient_b.actual_outcome  # 作为参考历史数据
            )

            # 计算预测轨迹与患者 B 实际轨迹的相似度
            trajectory_similarity = self.calculate_trajectory_similarity(
                cf_prediction_a,
                patient_b.actual_trajectory
            )

            # 计算置信度（基于 LLM 输出的确定性和医学合理性）
            confidence = self.calculate_prediction_confidence(cf_prediction_a)

            similarities.append({
                'treatment': treatment,
                'similarity': trajectory_similarity,
                'confidence': confidence,
                'rationale': cf_prediction_a.get('reasoning_chain', '')
            })

        RETURN similarities

    METHOD calculate_trajectory_similarity(predicted_traj, actual_traj):
        """计算两条时序轨迹的相似度"""
        # 处理不同长度和相位偏移的序列
        dtw_distance = DYNAMIC_TIME_WARPING(predicted_traj, actual_traj)

        # 变化模式
```

```
        shape_similarity = SHAPE_BASED_MATCHING(predicted_traj, actual_traj)

        # 临床重要事件的时间匹配度
        event_similarity = CLINICAL_EVENT_ALIGNMENT(predicted_traj, actual_traj)

        # 加权融合
        overall_similarity = 0.4*(1-dtw_distance) + 0.4*shape_similarity + 0.2*event_similarity
        RETURN overall_similarity
```

2.3 神经符号相似性度量模块

神经符号相似性度量模块通过融合数值计算和符号推理来评估患者之间的相似性，用于类比通道。首先计算数值相似性，捕捉生命体征和临床数据的相似程度；然后评估符号规则一致性，通过输入的规则检查患者是否遵循相同的诊疗路径，同时分析激活一些临床概念的模式是否相似，生成总体相似性分数与可解释的报告。

```
# 神经符号相似性度量
CLASS NeuroSymbolicSimilarity:
    # 初始化：加载临床规则库，配置相似性权重
    METHOD __init__(clinical_rules, similarity_config):
        self.clinical_rules = clinical_rules      # 诊疗指南、临床路径
        self.numeric_similarity = CosineSimilarity()  # 数值相似性计算
        self.symbolic_reasoner = RuleEngine()         # 符号规则推理引擎
        self.fusion_weights = similarity_config.weights  # 多维度权重

    # 多维度相似性计算
    METHOD compute_similarity(patient_a, patient_b, objective):
        similarity_components = {}

        # 1. 数值相似性：传统向量空间相似度
        similarity_components['numeric'] = COMPUTE_VECTOR_SIMILARITY(
            patient_a.embedding,
            patient_b.embedding
        )

        # 2. 符号规则一致性：检查临床指南符合度
```

```
        similarity_components['symbolic'] = evaluate_rule_consistency(
            patient_a, patient_b, self.clinical_rules
        )

        # 3. 概念空间相似性：比较临床概念激活模式
        similarity_components['conceptual'] = COMPARE_CONCEPT_ACTIVATIONS(
            patient_a.concept_activations,
            patient_b.concept_activations
        )

        # 4. 基于目标的加权融合
        weights = ADJUST_WEIGHTS_BY_OBJECTIVE(self.fusion_weights, objective)
        overall_similarity = WEIGHTED_AVERAGE(similarity_components, weights)

        # 5. 生成可解释报告
        explanation = GENERATE_EXPLANATION(similarity_components, weights)

        RETURN {
            'overall': overall_similarity,
            'components': similarity_components,
            'explanation': explanation
        }

    # 符号规则一致性评估
    METHOD evaluate_rule_consistency(patient_a, patient_b, rules):
        consistency_scores = []

        FOR each_rule IN rules:
            # 检查两个患者是否遵循相同的临床路径
            compliance_a = APPLY_RULE(patient_a, rule)
            compliance_b = APPLY_RULE(patient_b, rule)

            # 规则一致性 = 1 - 标准化差异
            consistency = 1 - NORMALIZED_DIFFERENCE(compliance_a, compliance_b)
            consistency_scores.append(consistency)

        RETURN AVERAGE(consistency_scores)

    # 生成可解释的相似性报告
```

```
    METHOD generate_explanation(components, weights):
        explanations = []

        IF components['numeric'] > 0.8:
            explanations.append("生命体征动态模式高度相似")
        ELIF components['numeric'] > 0.6:
            explanations.append("主要生理参数变化趋势一致")

        IF components['symbolic'] > 0.9:
            explanations.append("完全符合相同临床路径")
        ELIF components['symbolic'] > 0.7:
            explanations.append("遵循相似诊疗指南")

        IF components['conceptual'] > 0.85:
            explanations.append("疾病严重程度和类型高度匹配")

        # 基于权重的解释
        dominant_component = FIND_MAX_WEIGHT_COMPONENT(weights)
        explanations.append(f"相似性主要基于{dominant_component}分析")

        RETURN "; ".join(explanations)
```

2.4 联邦元学习匹配器

联邦元学习匹配器的任务是在隐私保护下，跨机构相似患者搜索。其通过联邦学习框架避免原始数据共享，以满足医疗数据的伦理要求。本模块在初始化时配置各医院的本地模型和全局元学习模型，并施加隐私噪声。在训练阶段，各医院下载全局模型进行本地训练以保证数据不出院，并添加噪声，再通过安全聚合算法全局更新，不断迭代优化元学习模型，从而提升相似性匹配的泛化能力。在搜索阶段，各医院本地计算相似性并匿名化候选结果，最终中心化排序返回 top-k 相似患者用于后续神经符号相似性度量。该方法可串联多个医院的数据，保护患者隐私以及医院利益，为系统提供可扩展且合规的相似病例检索功能。

```
# 联邦元学习匹配器
CLASS FederatedMetaMatcher:
    # 初始化：设置隐私参数
    METHOD __init__(hospital_configs, privacy_settings):
        self.hospitals = INITIALIZE_HOSPITALS(hospital_configs)
```

```
        self.global_model = MetaSimilarityModel()  # 全局元学习模型
        self.privacy_engine = DifferentialPrivacy(privacy_settings)

    # 联邦元训练：跨医院联合训练但不共享数据
    METHOD federated_meta_train():
        FOR round IN range(total_rounds):
            hospital_updates = []

            # 1. 各医院本地训练
            FOR each_hospital IN self.hospitals:
                # 下载全局模型
                hospital.download_model(self.global_model)

                # 本地训练（数据不出院）
                local_update = hospital.local_train()

                # 添加差分隐私噪声
                noisy_update = self.privacy_engine.add_noise(local_update)
                hospital_updates.append(noisy_update)

            # 2. 安全聚合更新
            global_update = SECURE_AGGREGATION(hospital_updates)

            # 3. 更新全局模型
            self.global_model.update(global_update)

    # 隐私保护下的相似患者搜索
    METHOD find_similar_patients(query_patient, top_k):
        all_candidates = []

        # 1. 编码查询患者（本地完成）
        query_embedding = query_patient.embedding  # 不共享原始数据

        # 2. 各医院本地计算相似性
        FOR each_hospital IN self.hospitals:
            # 隐私保护计算：只共享嵌入向量，不共享原始数据
            local_candidates = hospital.compute_local_similarities(query_embedding)

            # 匿名化处理
            anonymized_candidates = ANONYMIZE(local_candidates)
            all_candidates.extend(anonymized_candidates)

```

```
        # 3. 全局排序（中心化协调）
        sorted_candidates = SORT_BY_SIMILARITY(all_candidates)

        final_results = REFINE_RESULTS(sorted_candidates, top_k)

        RETURN final_results
```

2.5 整体工作流程

```
# 系统工作流程
METHOD find_most_similar_patients(query_patient, clinical_objective):
    # 步骤 1: 多模态编码
    encoded_patient = MultiscaleTemporalEncoder.encode(query_patient)

    # 步骤 2: 因果图构建
    causal_graph = CausalTemporalGraph_LLM.predict_counterfactual_with_llm(
        spatient_state, intervention, historical_outcomes
    )

    # 步骤 3: 联邦相似性搜索
    candidate_patients = FederatedMetaMatcher.find_similar_patients(
        encoded_patient.patient_embedding
    )

    # 步骤 4: 精细相似性分析
    detailed_results = []
    FOR candidate IN candidate_patients:
        # 神经符号相似性
        neuro_symbolic_sim = NeuroSymbolicSimilarity.compute_similarity(
            query_patient, candidate, clinical_objective
        )

        # 因果相似性
        causal_sim = CausalTemporalGraph_LLM.counterfactual_similarity_llm(
            query_patient, candidate, TREATMENT_OPTIONS
        )

        # 综合评分
        composite_score = COMBINE_SIMILARITIES(
```

```
            neuro_symbolic_sim, causal_sim, clinical_objective
        )

        detailed_results.append({
            'candidate': candidate,
            'composite_score': composite_score,
            'similarity_breakdown': {
                'neuro_symbolic': neuro_symbolic_sim,
                'causal': causal_sim
            },
            'clinical_rationale': GENERATE_CLINICAL_RATIONALE(
                neuro_symbolic_sim, causal_sim
            )
        })

    # 最终排序返回
    RETURN SORT_BY_COMPOSITE_SCORE(detailed_results)
```

3. 人机协同的记忆安全演化机制

人机协同的记忆安全演化机制针对记忆系统的动态演化需求，建立一套保守-渐进式记忆更新机制。该机制通过构建反馈价值评估模型，多维度量化指标触发记忆更新，并保存记忆演化轨迹，形成可回溯的决策知识演化链。

3.1 反馈价值评估模型

反馈价值评估模型从医生存储的决策轨迹日志中提取反馈事件（如干预后的患者状态变化），并基于四个维度计算价值分数：临床显著性、医生权威度、反馈频率以及信息增益度。模型输出一个综合价值分数（范围 0-1）并设定一个更新阈值，用于触发记忆更新。以下是对四个维度的介绍：

维度一：临床显著性

- 定义：反馈关联的生理参数变化是否具有统计学意义（如 p 值<0.05）。
- 所需信息：干预前后的患者生理指标（如 State 中 MAP、SOFA 评分）以及医生的决策轨迹。
- 存储位置：患者存储（PatientRecord 类）中的 State 字段：存储时间序列状态数据，包括生理参数；医生存储（DoctorProfile 类）中的 decision_trajectory：记录干预时间戳和患者 ID，可通过患者 ID 查询干预前后的状态变化。
- 计算方式：模型从 decision_trajectory 获取干预时间戳和患者 ID，然后查询患者存储的 State，提取干预前和干预后的参数值，进行统计检验。

维度二：医生权威度

- 定义：基于医生等级（职位）、专业领域匹配度、历史决策成功率。
- 所需信息：医生职位（如主治医师、主任医师）；医生专长（疾病名）与当前病例疾病诊断相匹配；医生历史决策成功率。
- 现有存储对应：医生存储（DoctorProfile 类）中的 personal_info：包含职位（position）、专长（specialty）和成功率（success_rate）；

- 计算方式：职位权重：预设权重（如主任医师=1.0，主治医师=0.7）；匹配度：比较医生专长（specialty）与患者诊断（diagnosis）；成功率：直接使用 success_rate 字段。最终将以上三个要素进行综合加权计算得到权威度。

维度三：反馈频率

- 定义：同类反馈在不同医生、不同病例中的重复出现频率。
- 所需信息：历史反馈事件库：包括干预类型、患者诊断、反馈结果
- 现有存储对应：
- 医生存储的 decision_trajectory：包含干预、患者 ID 和时间戳，但未集中存储所有反馈。
- 需要全局反馈日志：当前存储缺乏一个中央化的反馈存储，用于高效查询一致性。
- 计算方式：从全局反馈日志中，查询同类干预（如相同药物）在相似诊断病例中的出现次数。一致性分数 = 重复次数 / 总相关病例数。使用任务二的神经符号相似性模块（NeuroSymbolicSimilarity）来定义“相似病例”（基于患者嵌入和诊断）。

维度四：信息增益度

- 定义：反馈是否能弥补知识空白或纠正问题，通过信息熵变化测量。
- 所需信息：知识存储的当前状态，如规则熵；反馈内容（如新规则或规则修正）。
- 现有存储对应：clinical_rules：存储现有规则。反馈事件：从 decision_trajectory 或全局反馈日志中提取反馈内容。
- 计算方式：基于规则的不确定性，计算知识库当前熵。并计算执行反事实干预后的熵变化。
- 信息增益 = 原始熵 - 新熵。

在反馈价值评估模型识别出高价值反馈后，通过保守-渐进式协议安全地更新记忆系统，并全程追踪演化轨迹。方案确保在安全性与有效性间建立平衡。该系统采用分层更新架构，利用三种前沿技术实现不同场景的记忆更新任务：针对情

况紧急的临床修正时间，如 ICU 剂量错误，通过 ROME 技术实现毫秒级参数级精准编辑，仅修改特定医学概念，保障实时性的同时避免知识遗忘；对常规知识如指南更新，采用 MOOM 双阶段机制——日间创建待验证记忆单元供临床参考，夜间通过聚类分析与医学规则校验实现安全整合；所有更新由 MEMORYLLM 以可解释的临床知识单元统一管理，通过优先级缓冲池和版本化替换实现渐进式知识更新。该系统在安全性（MOOM 离线条文验证）、实时性（ROME 紧急通道）与可追溯性（MEMORYLLM 结构化审计）间建立平衡，为脓毒症治疗提供动态演进的可靠决策支持。伪代码示例如下：

```
// 系统初始化
System INIT:
    MOOM = NEW MOOM_Updater()
    ROME = NEW ROME_Editor()
    MemoryLLM = NEW MemoryLLM_Manager()
    Evaluator = NEW FeedbackEvaluator()

// 主处理流程
PROCEDURE ProcessFeedback(FeedbackEvent):
    // 价值评估
    Evaluation = Evaluator.evaluate(FeedbackEvent)

    IF Evaluation.score >= 0.9 AND FeedbackEvent.urgency == "CRITICAL":
        // 紧急更新通道 (ROME)
        Result = ExecuteEmergencyUpdate(FeedbackEvent)
    ELSE:
        // 常规更新通道 (MOOM+MemoryLLM)
        Result = ExecuteStandardUpdate(FeedbackEvent, Evaluation)
    RETURN Result

// 紧急更新 (ROME)
PROCEDURE ExecuteEmergencyUpdate(FeedbackEvent):
    // 定位关键参数层
    TargetLayer = ROME.locate_parameter_layer(FeedbackEvent.concept)

    // 执行最小秩编辑
    UpdateResult = ROME.apply_rank_one_edit(
        layer = TargetLayer,
        new_value = FeedbackEvent.content
    )
    RETURN {"status": "URGENT_UPDATE", "result": UpdateResult}

```

```
// 常规更新 (MOOM+MemoryLLM)
PROCEDURE ExecuteStandardUpdate(FeedbackEvent, Evaluation):
    // 创建临时记忆单元
    TempToken = MOOM.create_provisional_entry(FeedbackEvent)

    // 存入缓冲池
    MemoryLLM.add_to_buffer(TempToken, Evaluation.score)
    RETURN {"status": "PENDING_VERIFICATION", "token_id": TempToken.id}

// 夜间批处理
PROCEDURE NightlyBatchProcess():
    // MOOM 离线验证
    VerifiedUpdates = MOOM.validate_and_cluster()

    // MemoryLLM 整合
    FOR EACH Update IN VerifiedUpdates:
        MemoryLLM.integrate_token(Update.token)

    // 生成审计报告
    Report = MOOM.generate_audit_report(VerifiedUpdates)
    RETURN Report
```